PLFA in Lean

Daniel

Contents

1 Part 1

This part collects the early material on natural numbers and induction.

1.1 Naturals

Most are familiar with the natural numbers:

```
0
1
2
3
...
```

and so on. The set of natural numbers `ℕ` is inifite and we say that `0`, `1`, `2`, `3`, and are **values** of type `ℕ` (otherwise indicated with: `0 : ℕ`, `1 : ℕ`, etc.).

Although there are infinitely many naturals, yet we can write down its definition in just a few lines. Here it is as a pair of inference rules:

```
--------
zero : ℕ

m : ℕ
--------
suc m : ℕ
```

And here is the definition in Lean:

```
inductive ℕ : Type where
    | zero : ℕ
    | suc  : ℕ -> ℕ
```

Here `ℕ` is the name of the datatype we are defining, and `zero` and `suc` (short for successor) are the **constructors** of the datatype.

Both definitions above tell us two things:

1. base case: `zero` is a natural number
2. inductive case: if `m` is a natural number, then `suc m` is also a natural number.

Further, these two rules give the **only** ways of creating natural numbers. Hence, the possible natural numbers are:

```
.zero
.suc .zero
.suc (.suc .zero)
...
```

We write `0` as the shorthand for the `zero` constructor and `1` as shorthand for `suc zero`, the successor of zero, that is the number that comes after `0`.

Exercise: `seven` (practice)

Write out `7` in longhand.

potential sol:

```
def seven : ℕ :=
  .suc $ .suc $ .suc $ .suc $ .suc $ .suc $ .suc .zero
```

Unpacking the inference rules

Let's unpack the Lean definition. the keyword `inductive` tells us this is an inductive definition, that is, we are defining a new datatype with constructors. The phrase:

```
ℕ : Type
```

tells us that `ℕ` is the name of the new data, type and that it is a `Type`, which is the way in Lean of saying that it is a type.

The keyword `where` is the name of the new datatype, and that is a `Type`, which is the way in Lean of saying that it is a type. The keyword `where` separates the declaration of the data from the declaration of its constructors. Each constructor is a declared on a separate line, which is indented to indicate that it belongs to the corresponding `data` declaration. The lines:

```
zero : ℕ
suc  : ℕ -> ℕ
```

give *signatures* specifying the types of the constructors `zero` and `suc`. They tell us `zero` is a natural number and that `suc` takes a natural number as an argument and returns a natural number.

You may have noticed that ℕ and → don't appear on your keyboard. They are symbols in *unicode*. At the end of each chapter is a list of all unicode symbols introduced in the chapter, including instruction on how to insert them into the editor (assuming you're using zed or intellij).

Lean4 Natural Literal Syntax

Lean4 already defines its own inductive type for the natural numbers and also supports integer literal syntax. We can hook into that sytax and relate it to our own ℕ type with the following conversion function and typeclass instance implementation:

```
def convert : _root_.Nat -> ℕ
    | _root_.Nat.zero   => .zero
    | _root_.Nat.succ n => .suc (convert n)

instance (n : _root_.Nat) : OfNat ℕ n where
    ofNat := convert n
```

Operations on naturals are recursive functions

Now that we have the natural numbers, what can we do with them? For instance, can we define arithmetic operations such as addition and multiplication?

It turns out, each of these can be described as *recursive* functions. Here is the definition of addition for the ℕ type in lean4:

```
def plus : ℕ -> ℕ -> ℕ
    | .zero , n    => n
    | (.suc m) , n => .suc (plus m n)

syntax:65 (priority := high) term:66 " + " term:65 : term

macro_rules
  | `($a + $b) => `(plus $a $b)
```

The first line of `plus` gives its type: `ℕ -> ℕ -> ℕ`, which indicates that the `plus` function accepts two naturals and returns a natural. The infix notation allows us to write plus using the usual infix + notation. The `priority := high` bit makes it more likely that uses of + will get resolved to our custom `plus` function (for our own natural number type (ℕ) based on the surrounding context).

If we write zero as `0` and `suc m` as `1 + m`, the definition turns into two familiar equations:

```
 0       + n  =  n
 (1 + m) + n  =  1 + (m + n)
```

The first follows because zero is an identity for addition, and the second because addition is associative. In its most general form, associativity is written

```
(m + n) + p = m + (n + p)
```

meaning the location of the parentheses is irrelevant.

An example of how we can use these constructors (better more illustrative example second I think):

```
example : 3 + 2 = 5 :=
  calc
      3 + 2
      = .suc (2 + 2)                 := rfl
      _ = .suc (.suc (1 + 2))        := rfl
      _ = .suc (.suc (.suc (0 + 2))) := rfl
      _ = .suc (.suc (.suc 2))       := rfl
      _ = 5                          := rfl
```

Note we could've expressed the theorem like so if we wanted to make it more clear in the local context that these literals are formed via our `ℕ` type like so: `example : 3 + 2 = (5 : ℕ)`

Here's another version closer to the original PLFA text:

```
section
open ℕ  -- so we don't need dots in front of each ctor
example : 2 + 3 = 5 :=
  calc
      2 + 3
      = (suc (suc zero)) + (suc (suc (suc zero)))   := rfl
      _ = suc ((suc zero) + (suc (suc (suc zero)))) := rfl
      _ = suc (suc (zero + (suc (suc (suc zero))))) := rfl
      _ = suc (suc (suc (suc (suc zero))))          := rfl
      _ = 5                                         := rfl
end
```

Multiplication

Once we have defined addition, we can define multiplication as repeated addition:

```
def mult : ℕ -> ℕ -> ℕ
    | .zero, n      => .zero
    | (.suc m), n   => n + (mult m n)
```

```
-- infix syntax rules for infix syntax:
syntax:70 (priority := high) term:70 " * " term:71 : term

macro_rules
  | `($a * $b) => `(mult $a $b)
```

Computing `m * n` returns the sum of `m` copies of `n`. Again, rewriting turns the definition into two familiar equations:

```
0       * n  =  0
(1 + m) * n  =  n + (m * n)
```

Exercise `*-example` (practice)

Compute `3 * 4` writing out your reasoning as a chain of equations using the equations for `*`. You do not need to step through the evaluation of `+`

potential sol:

```
example : 3 * 4 = 12 :=
    calc
        3 * 4
      = 4 + (2 * 4)              := rfl
    _ = 4 + (4 + (1 * 4))        := rfl
    _ = 4 + (4 + (4 + (0 * 4)))  := rfl
    _ = 4 + (4 + (4 + 0))        := rfl
    _ = 4 + 4 + 4                := rfl
    _ = 12                       := rfl
```

Exercise exponentiation (recommended)

Define exponentiation, which is given by the following equations:

```
m ^ 0           = 1
m ^ (1 + n)     = m * (m^n)
```

potential sol:

```
def exponent : ℕ -> ℕ -> ℕ
    | m , 0         => 1
    | m , (.suc n)  => m * (exponent m n)

syntax:80 (priority := high) term:81 " ^ " term:80 : term

macro_rules
  | `($a ^ $b) => `(exponent $a $b)
```

Check that `3 ^ 4` is 81:

```
#check (3 ^ 4 = 81)
```

Monus

We can also define subtraction for natural numbers. Since there are no negative natural numbers, if we subtract a larger number from a smaller number we will take the result to be zero. This adaption of subtraction to naturals is called monus (a twist on *minus*).

Monus is our first use of a definition that uses pattern matching against both arguments:

```
def monus : ℕ -> ℕ -> ℕ
    | m, .zero              => m
    | .zero, (.suc n)       => .zero
    | (.suc m), (.suc n)    => monus m n

syntax:80 (priority := high) term:81 " ∸ " term:80 : term

macro_rules
  | `($a ∸ $b) => `(monus $a $b)
```

where lean performs case analysis to ensure that all the cases are covered.

- consider the second argument:
- – if it is `zero`, then the first equation applies
- – if it is `suc n`, then consider the first argument
- – a) if it is `zero`, then the second equation applies
- – b) if it is `suc m`, then the third equation applies

Lean will raise an error if all the cases are not covered. As with addition and multiplication, the recursive definition is well founded because monus on bigger numbers is defined in terms of monus on smaller numbers.

For example, let's subtract two from three:

```
example : 3 ∸ 2 = 1 :=
    calc
        2 ∸ 1
      = 1 ∸ 0 := rfl
    _ = 1     := rfl
```

We did not use the second equation at all, but it will be required if we try to subtract a larger number from a smaller one:

```
example : 2 ∸ 3 = 0 :=
    calc
        1 ∸ 2
    _ = 0 ∸ 1        := rfl -- desugars to:
    _ = 0 ∸ (.suc 0) := rfl
    _ = 0            := rfl
```

Currying

We have chosen to represent a function of two arguments in terms of a function of the first argument that returns a function of the second argument. This trick goes by the name *currying*.

Lean4, like other functional languages such as Haskell and ML, is designed to make currying easy to use. Function arrows associate to the right and application associates to the left:

`ℕ -> ℕ -> ℕ` associates like so: `ℕ -> (ℕ -> ℕ)`

and

`plus 2 3` stands for `(plus 2) 3`

The term `plus 2` by itself stands for the function that adds two to its argument, hence applying it to three yields five.

Currying is named for Haskell B. Curry (a Penn State professor actually: https://www.hmdb.org/m.asp?m=134735; bio[1])

Story of creation, revisited

Just as our inductive definition defines the naturals in terms of the naturals, so does our recursive definition define addition in terms of addition.

Again we resort to a creation story, where this time we are concerned with judgments about addition:

```
-- In the beginning, we know nothing about addition.
```

Now, we apply the rules to all the judgment we know about. The base case tells us that `.zero + n = n` for every natural `n`, so we add all those equations. The inductive case tells us that if `m + n = p` (on the day before today) then `suc m + n = suc p` (today). We didn't know any equations about addition before today, so that rule doesn't give us any new equations:

```
-- On the first day, we know about addition of 0.
0 + 0 = 0     0 + 1 = 1    0 + 2 = 2     ...
```

[1] `https://mathshistory.st-andrews.ac.uk/Biographies/Curry/`

Then we repeat the process, so on the next day we know about all the equations from the day before, plus any equations added by the rules. The base case tells us nothing new, but now the inductive case adds more equations:

```
-- On the second day, we know about addition of 0 and 1.
0 + 0 = 0     0 + 1 = 1     0 + 2 = 2     0 + 3 = 3     ...
1 + 0 = 1     1 + 1 = 2     1 + 2 = 3     1 + 3 = 4     ...
```

And we repeat the process again:

```
-- On the third day, we know about addition of 0, 1, and 2.
0 + 0 = 0     0 + 1 = 1     0 + 2 = 2     0 + 3 = 3     ...
1 + 0 = 1     1 + 1 = 2     1 + 2 = 3     1 + 3 = 4     ...
2 + 0 = 2     2 + 1 = 3     2 + 2 = 4     2 + 3 = 5     ...
```

You've got the hang of it by now:

```
-- On the fourth day, we know about addition of 0, 1, 2, and 3.
0 + 0 = 0     0 + 1 = 1     0 + 2 = 2     0 + 3 = 3     ...
1 + 0 = 1     1 + 1 = 2     1 + 2 = 3     1 + 3 = 4     ...
2 + 0 = 2     2 + 1 = 3     2 + 2 = 4     2 + 3 = 5     ...
3 + 0 = 3     3 + 1 = 4     3 + 2 = 5     3 + 3 = 6     ...
```

The process continues. On the m'th day we will know all the equations where the first number is less than `m`.

As we can see, the reasoning that justifies inductive and recursive definitions is quite similar. They might be considered two sides of the same coin.

The story of creation, finitely

The above story was told in a stratified way. First, we create the infinite set of naturals. We take that set as given when creating instances of addition, so even on day one we have an infinite set of instances.

Instead, we could choose to create both the naturals and the instances of addition at the same time. Then on any day there would be only a finite set of instances:

```
-- In the beginning, we know nothing.
```

Now, we apply the rules to all the judgment we know about. Only the base case for naturals applies:

```
-- On the first day, we know zero.
0 : ℕ
```

Again, we apply all the rules we know. This gives us a new natural, and our first equation about addition.

```
-- On the second day, we know one and all sums that yield zero.
0 : ℕ
1 : ℕ    0 + 0 = 0
```

Then we repeat the process. We get one more equation about addition from the base case, and also get an equation from the inductive case, applied to equation of the previous day:

```
-- On the third day, we know two and all sums that yield one.
0 : ℕ
1 : ℕ    0 + 0 = 0
2 : ℕ    0 + 1 = 1   1 + 0 = 1
```

You've got the hang of it by now:

```
-- On the fourth day, we know three and all sums that yield two.
0 : ℕ
1 : ℕ    0 + 0 = 0
2 : ℕ    0 + 1 = 1   1 + 0 = 1
3 : ℕ    0 + 2 = 2   1 + 1 = 2    2 + 0 = 2
```

On the n'th day there will be `n` distinct natural numbers, and `n × (n-1) / 2` equations about addition. The number `n` and all equations for addition of numbers less than `n` first appear by day `n+1`. This gives an entirely finitist view of infinite sets of data and equations relating the data.

Exercise `Bin` (stretch)

A more efficient representation of natural numbers uses a binary rather than a unary system. We represent a number as a bitstring:

```
inductive Bin : Type where
    | empty  : Bin
    | t_zero : Bin -> Bin
    | t_one  : Bin -> Bin
    deriving Repr

syntax:80 (priority := high) "⟨⟩" : term
```

```
syntax:81 term:80 "O" : term
syntax:81 term:80 "I" : term
macro_rules
  | `(⟨⟩)        => `(Bin.empty)
  | `($a:term O) => `(Bin.t_zero $a)
  | `($a:term I) => `(Bin.t_one $a)
```

So the bitstring:

```
1011
```

would stand for the number eleven, encoded in our meta-introduced notation like so:

```
#eval ⟨⟩ I O I I
```

written desugared:

```
Bin.t_one (Bin.t_one (Bin.t_zero (Bin.t_one (Bin.empty))))
```

Representations are not unique due to leading zeroes. Hence eleven can also be represented by `0001011`, encoded as:

```
#eval ⟨⟩ O O I O I I
```

Define a function:

```
def inc : Bin -> Bin
```

that converts a bitstring to the bitstring for the next higher number.

1.2 Induction: Proof by Induction

Now that we've defined the naturals and operations on them, our next step is to learn how to prove properties that they satisfy. As hinted by their name, properties of *inductive datatypes* are proved by *induction*.

Properties of operators

Operations popup all the time, and mathematicians have agreed on names for some of the most common properties:

- *Identity*. Operator + has left identity `0` if `0 + n = n`, and right identity if `n + 0 = n`, for all `n`.